DIGITAL IMAGE PROCESSING · LECTURE 2

Smoothing: Low-Pass Spatial Filtering

From plain averaging to filters that look at what they are averaging

Digital Image Fundamentals · Signal Processing

ROADMAP

Today's Agenda

PART I

Mean (Box) Filtering

Neighbourhood averaging · padding · blur vs. window size

PART II

Gaussian & Binomial Filtering

Gaussian coefficients in the window · exact values · binomial approximation

PART III

Why This Is Low-Pass Filtering

What averaging does to slow and fast variation · measured response

PART IV

Bilateral Filtering

Adding an intensity term · parameters · denoising and detail enhancement

PART V

Non-Local Means

Self-similarity · patch-based weights · results and extensions

PART I

Mean (Box) Filtering

The simplest neighbourhood operation: local averaging

From Point Processing to Neighbourhoods

Lecture 1: point processing

$$s(x,y) = T[\, r(x,y) \,]$$

Output depends only on the single input pixel at that location.

Today: neighbourhood processing

$$s(x,y) = T[\, \text{window around } (x,y) \,]$$

Output depends on a whole window of pixels around that location.

Slide a square window over the image. At each position, combine the pixels inside it into one output value.

FOUNDATIONS

Padding: What Happens at the Border?

Near the edges, part of the window falls outside the image. Padding decides what value to use out there.

Zero padding

Treat outside pixels as 0. Simple, but creates a dark border after smoothing.

Replicate

Repeat the nearest edge pixel outward. Avoids the dark-border artifact.

Reflect

Mirror the image across its own edge. Often the smoothest-looking result.

The Box (Mean) Filter

An m x n window in which every coefficient is equal, normalized so they sum to 1.

3 x 3 box window

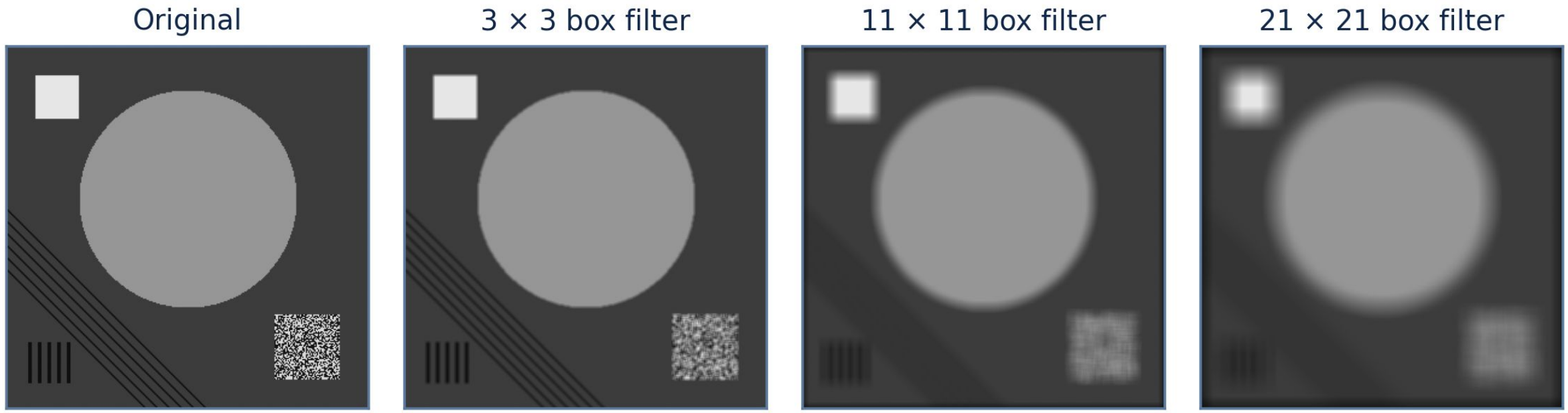
1/9	1/9	1/9
1/9	1/9	1/9
1/9	1/9	1/9

$$w(s,t) = 1 / (mn)$$

The output is the plain local average — a running mean over the window.

Normalizing keeps a flat region's intensity unchanged: average a patch of constant grey and you get the same grey back.

Mean Filtering in Action: Window Size vs. Blur



Larger windows blur more. The thin lines and fine noise patch — detail comparable in size to the window — are wiped out first; the large circle survives even 21x21 largely intact.

Limitations of the Box Filter

Every pixel counts equally

A pixel at the far corner of the window pulls on the result exactly as hard as the one right next to the centre. Intuitively, nearby pixels should matter more.

Abrupt window boundary

A pixel just inside the window has full influence; one pixel further out has none at all. Shifting the window by one pixel can change the output sharply.

Blind to content

The same flat average is applied everywhere — including across a strong edge, where the two sides have nothing to do with each other.

PART II

Gaussian & Binomial Filtering

Same square window — but the coefficients now fall off with distance

Gaussian Coefficients in the Window

Keep exactly the same square window as the box filter. Only the coefficients change: instead of all equal, they decay with distance from the centre.

$$w(s,t) = K \cdot \exp\left(-(s^2 + t^2) / 2\sigma^2 \right)$$

s, t = offsets from the window centre; K normalizes the coefficients to sum to 1

Nearby pixels count more

Influence tapers off with distance instead of stopping abruptly at the window edge.

σ controls the amount of blur

A larger σ makes the coefficients decay more slowly, so more of the window contributes.

Still a square window

Same sliding window, same arithmetic as the box filter — only the numbers inside differ.

Exact Gaussian Coefficients for Several Window Sizes

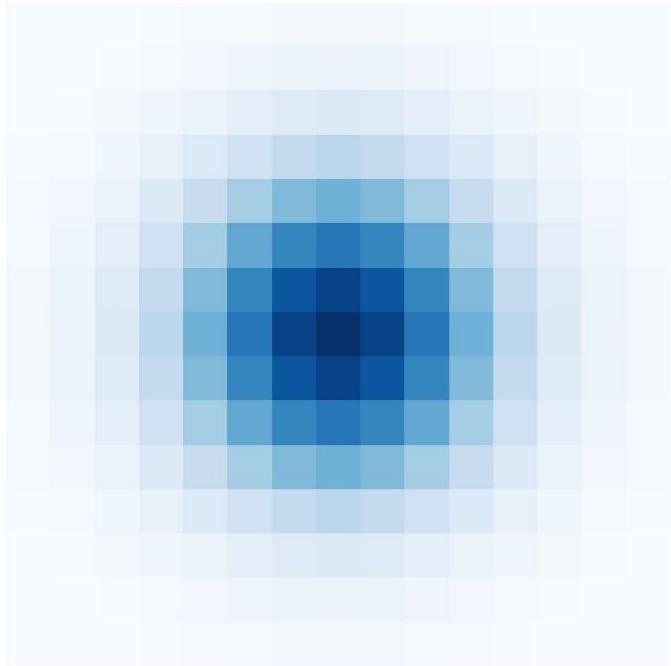
$5 \times 5 \quad (\sigma = 0.83)$

0.001	0.006	0.013	0.006	0.001
0.006	0.054	0.112	0.054	0.006
0.013	0.112	0.230	0.112	0.013
0.006	0.054	0.112	0.054	0.006
0.001	0.006	0.013	0.006	0.001

$9 \times 9 \quad (\sigma = 1.50)$

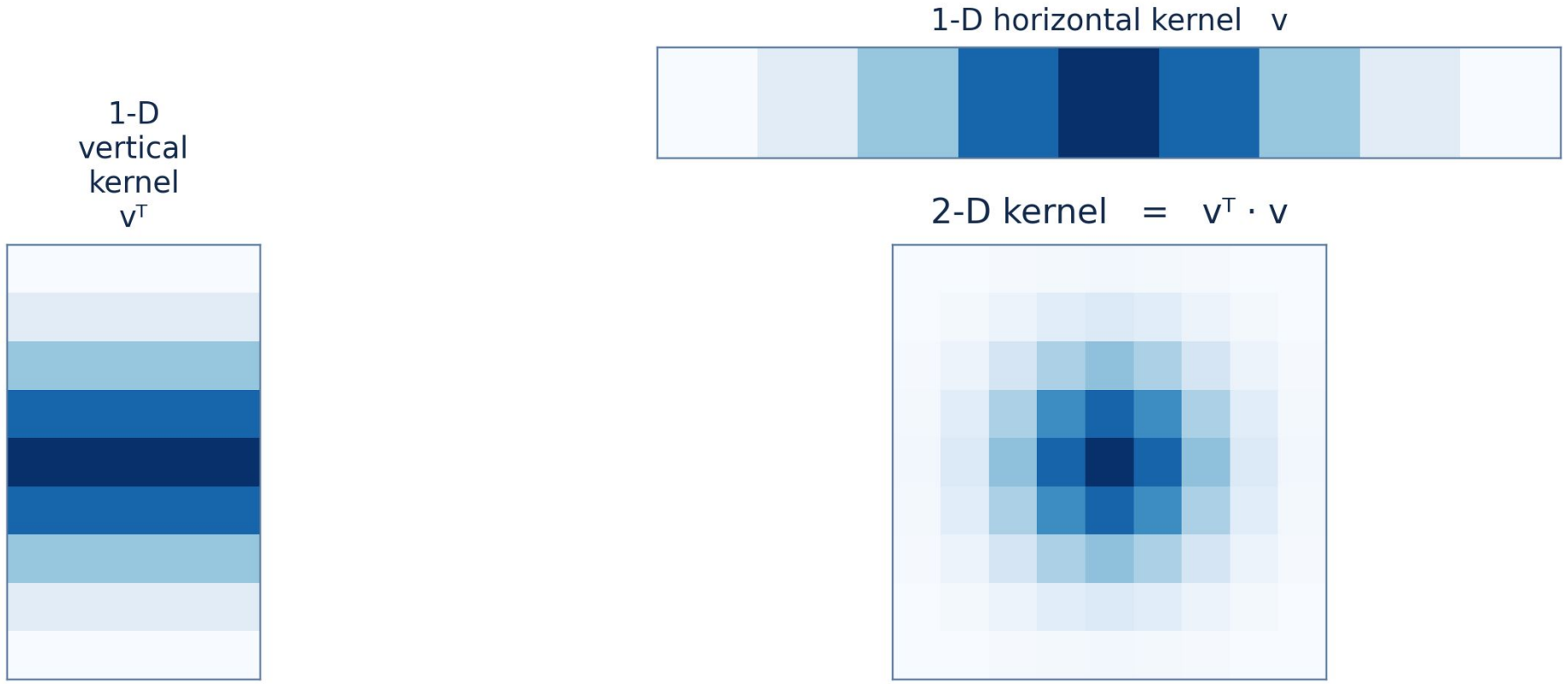
0.000	0.000	0.001	0.002	0.002	0.002	0.001	0.000	0.000
0.000	0.001	0.004	0.008	0.010	0.008	0.004	0.001	0.000
0.001	0.004	0.012	0.023	0.029	0.023	0.012	0.004	0.001
0.002	0.008	0.023	0.046	0.057	0.046	0.023	0.008	0.002
0.002	0.010	0.029	0.057	0.071	0.057	0.029	0.010	0.002
0.002	0.008	0.023	0.046	0.057	0.046	0.023	0.008	0.002
0.001	0.004	0.012	0.023	0.029	0.023	0.012	0.004	0.001
0.000	0.001	0.004	0.008	0.010	0.008	0.004	0.001	0.000
0.000	0.000	0.001	0.002	0.002	0.002	0.001	0.000	0.000

$15 \times 15 \quad (\sigma = 2.50)$



Evaluate the Gaussian formula at every integer offset in the window, then divide by the total so the coefficients sum to 1. These are the actual numbers you would type into the filter.

Separability: Two 1-D Passes Instead of One 2-D Pass



The 2-D coefficient table is the outer product of a 1-D row with itself. So filtering the rows and then the columns gives exactly the same answer as the full 2-D window — at a fraction of the arithmetic.

BEYOND THE TEXTBOOK

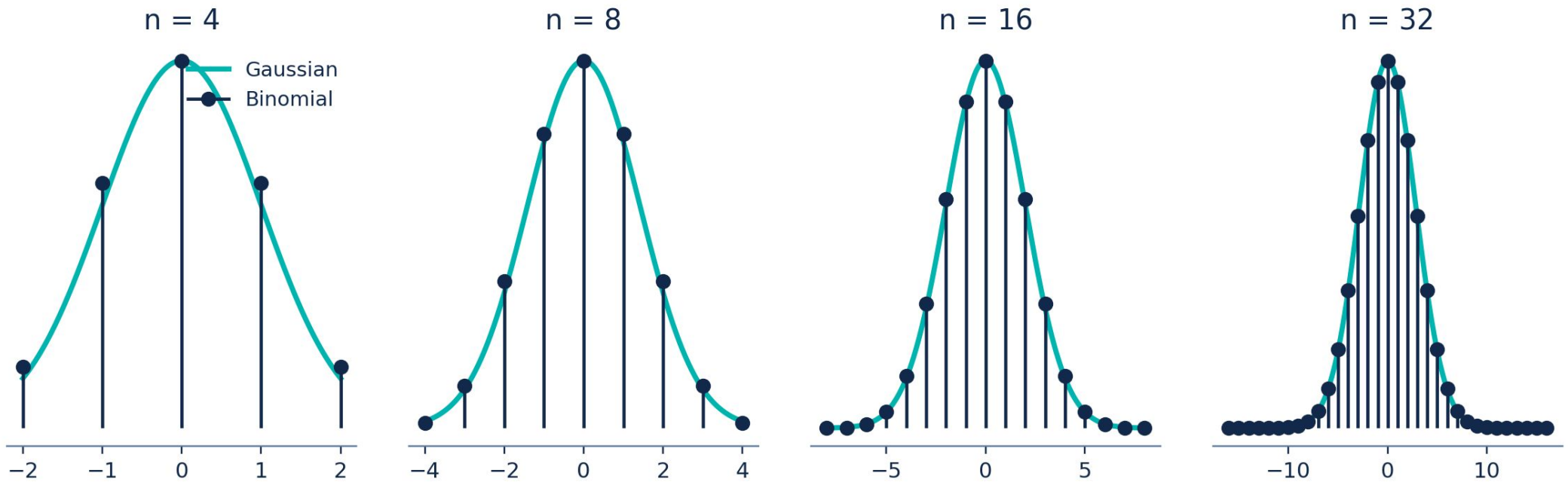
Approximating the Gaussian by Binomials

Not in Gonzalez — but a classic, cheap alternative. The rows of Pascal's triangle, normalized, are discrete stand-ins for Gaussian coefficients.

						1									n=0 ÷ 1
					1		1								n=1 ÷ 2
				1		2		1							n=2 ÷ 4
			1		3		3		1						n=3 ÷ 8
		1		4		6		4		1					n=4 ÷ 16
	1		5		10		10		5		1				n=5 ÷ 32
1		6		15		20		15		6		1			n=6 ÷ 64

By the Central Limit Theorem, a Binomial(n, ½) distribution approaches a Gaussian shape as n grows — and these rows are exactly that distribution.

How Good Is the Approximation?



Binomial coefficients (dots) against the variance-matched Gaussian (curve). Even a modest n = 16 or 32 is already hard to tell apart from the exact Gaussian.

The 2-D Binomial Window

5×5 binomial kernel × 256
(outer product of 1,4,6,4,1)

1	4	6	4	1
4	16	24	16	4
6	24	36	24	6
4	16	24	16	4
1	4	6	4	1

Same separability trick

The 2-D table is the outer product of the 1-D row with itself — exactly as with the Gaussian.

Integer-only arithmetic

No floating point: multiply, add, then divide by a power of two — a bit-shift. Ideal for cheap or fixed-point hardware.

A familiar kernel

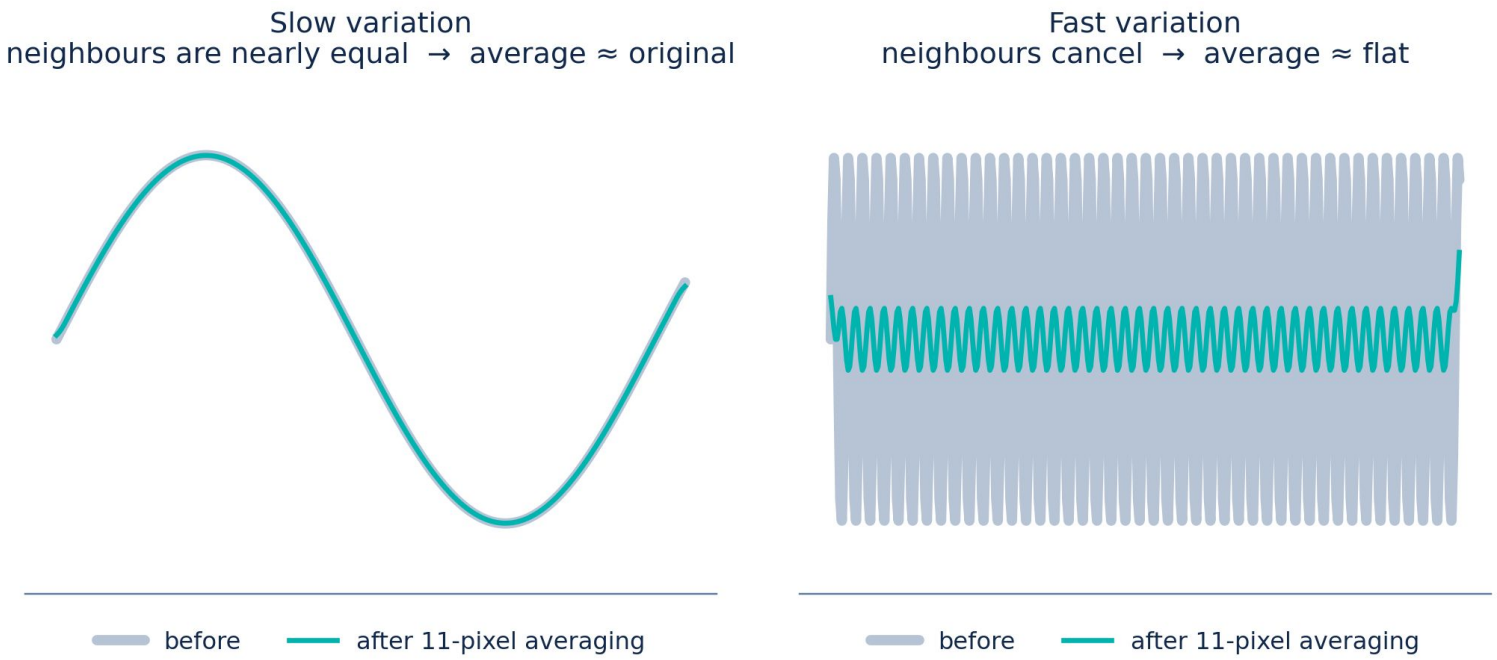
This 5x5 table is the one used in the Burt–Adelson image pyramid, throughout multi-resolution processing.

PART III

Why This Is Low-Pass Filtering

What averaging actually does to slow and fast variation

What Averaging Does to Slow and Fast Variation



Where the image changes slowly, the neighbours inside the window are all nearly the same value — so their average is nearly that value, and the signal comes through untouched. Where it changes quickly, the window holds roughly as many high values as low ones, and they cancel.

That Is Exactly What "Low-Pass" Means

Slow variation → survives

Broad regions, gentle shading, the overall layout of the scene — passed through.

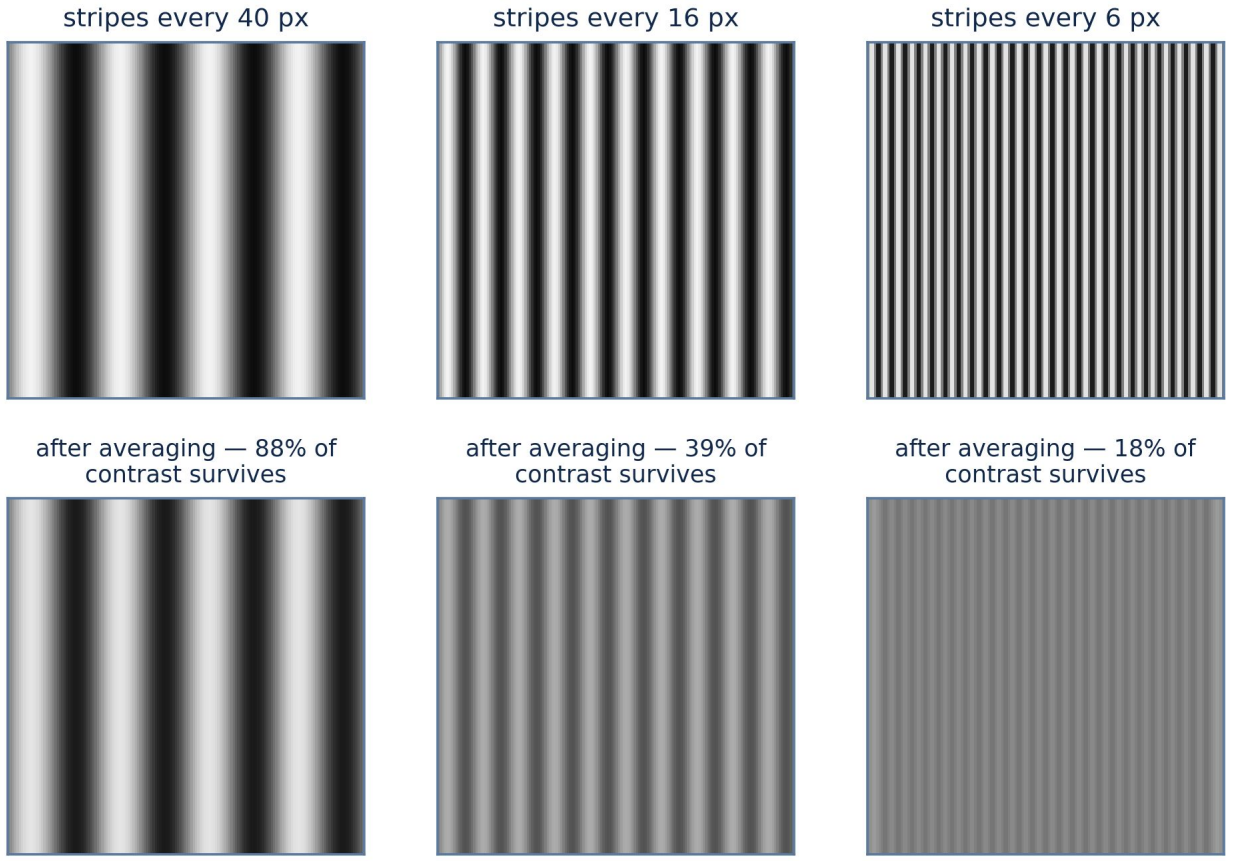
Fast variation → suppressed

Fine texture, thin lines, speckle noise — flattened out.

A filter that keeps slow variation and removes fast variation is called low-pass.

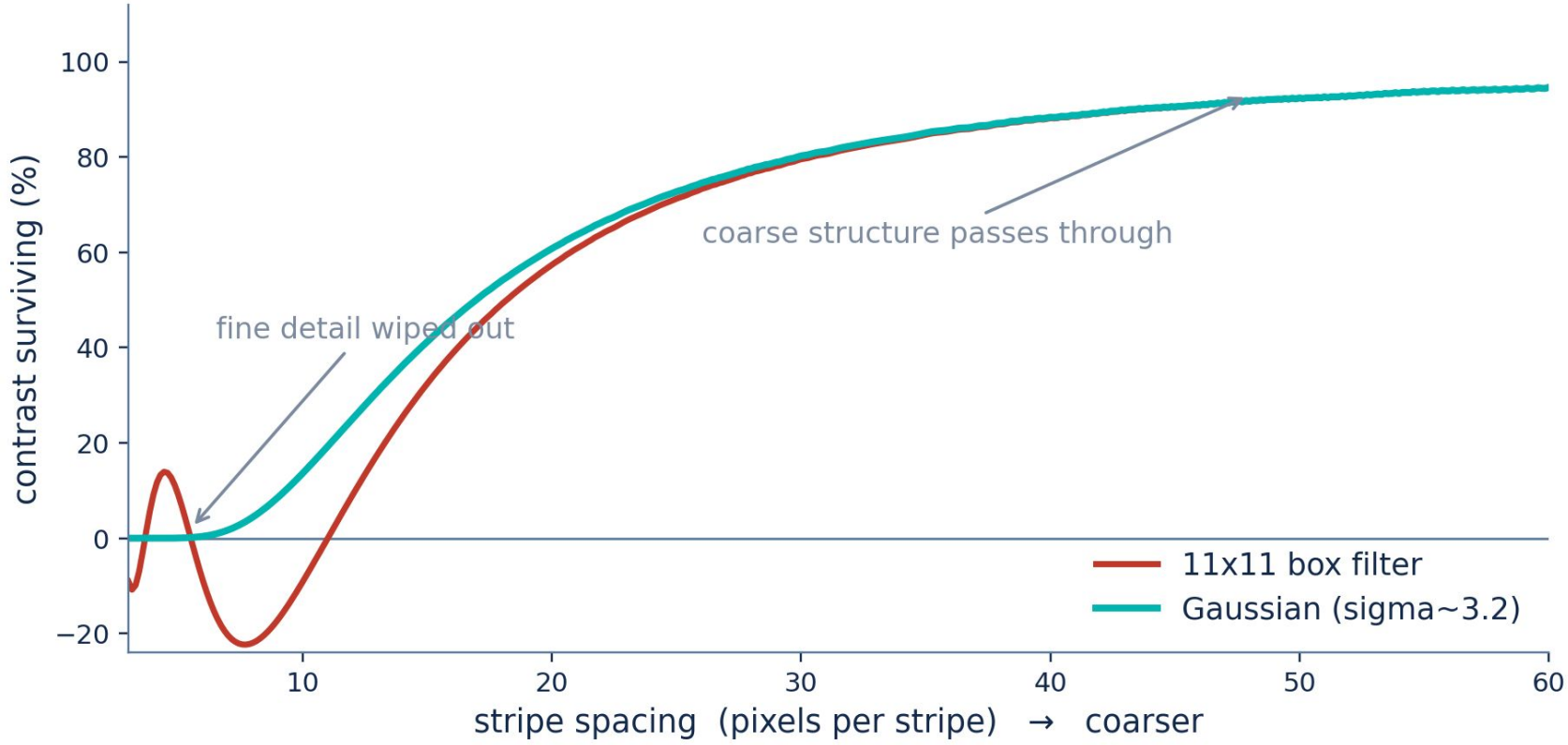
Nothing here required a change of representation — it follows directly from what a local average does.

Probing the Filter with Striped Patterns



Feed the filter test patterns that vary at a known rate and measure how much contrast comes out the other side. Coarse stripes barely change; fine stripes are almost erased.

The Measured Response of Each Filter



Sweeping the stripe spacing traces out each filter's response. The Gaussian falls off smoothly to zero. The box filter is messier: it dips below zero, meaning some fine patterns come back out with their light and dark bands swapped.

WHERE THIS BREAKS DOWN

The Blind Spot They All Share

Box, Gaussian and binomial differ only in the numbers inside the window. None of them ever looks at the pixel values it is averaging.

An edge is fast variation

A real boundary between two objects changes fast — so a low-pass filter treats it exactly like noise and smears it.

The window straddles both sides

At a pixel next to an edge, the window covers pixels from the other object entirely unrelated to it, and averages them in anyway.

The fix

Let the weights depend on the image itself, not just on position. That single change gives us the bilateral filter.

PART IV

Bilateral Filtering

Weights that depend on the image, not just on position

DEFINITION

The Bilateral Filter

Still a weighted average of the pixels in a window. But every neighbour is now judged on two things at once.

$$w(p,q) = \exp\left(-\|p-q\|^2 / 2\sigma_s^2\right) \times \exp\left(-\left(I(p)-I(q)\right)^2 / 2\sigma_r^2\right)$$

Space term (σ_s)

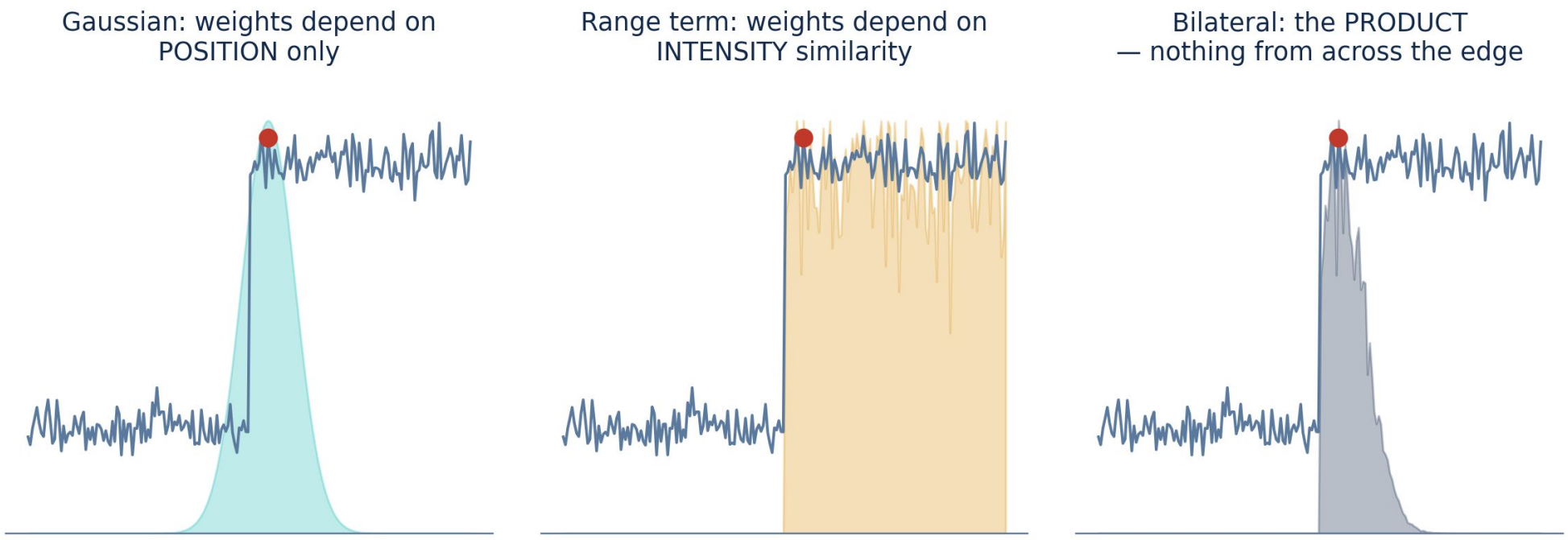
How far away is the neighbour?
The familiar Gaussian falloff — nothing new.

Range term (σ_r)

How different is its intensity?
This is the new part — and it reads the image.

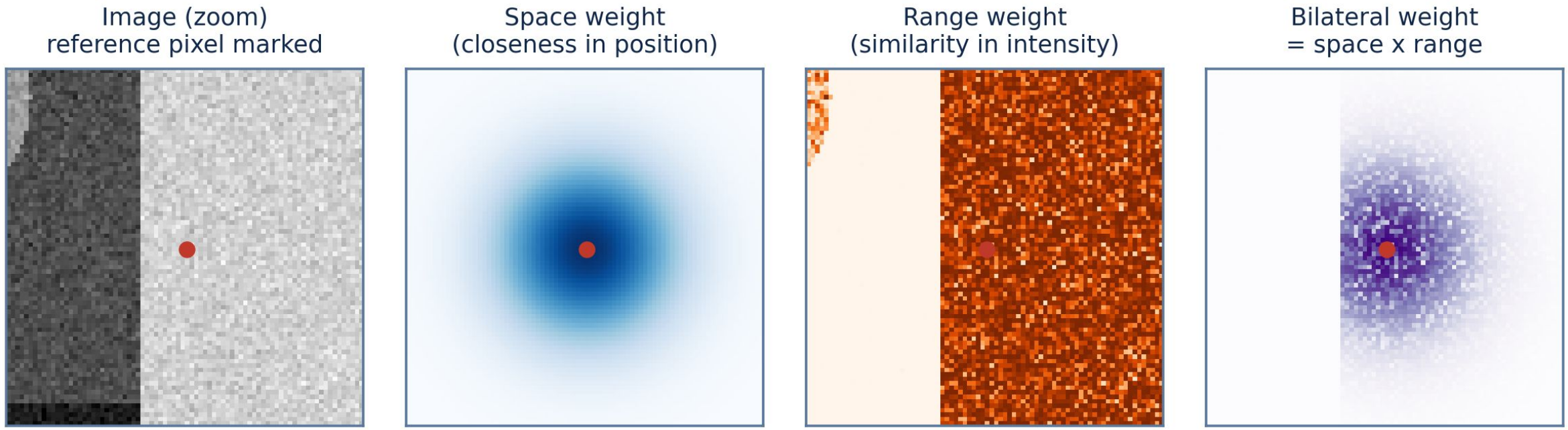
Aurich & Weule 1995 · Smith & Brady 1997 · Tomasi & Manduchi 1998

The Two Terms, Seen on a 1-D Signal



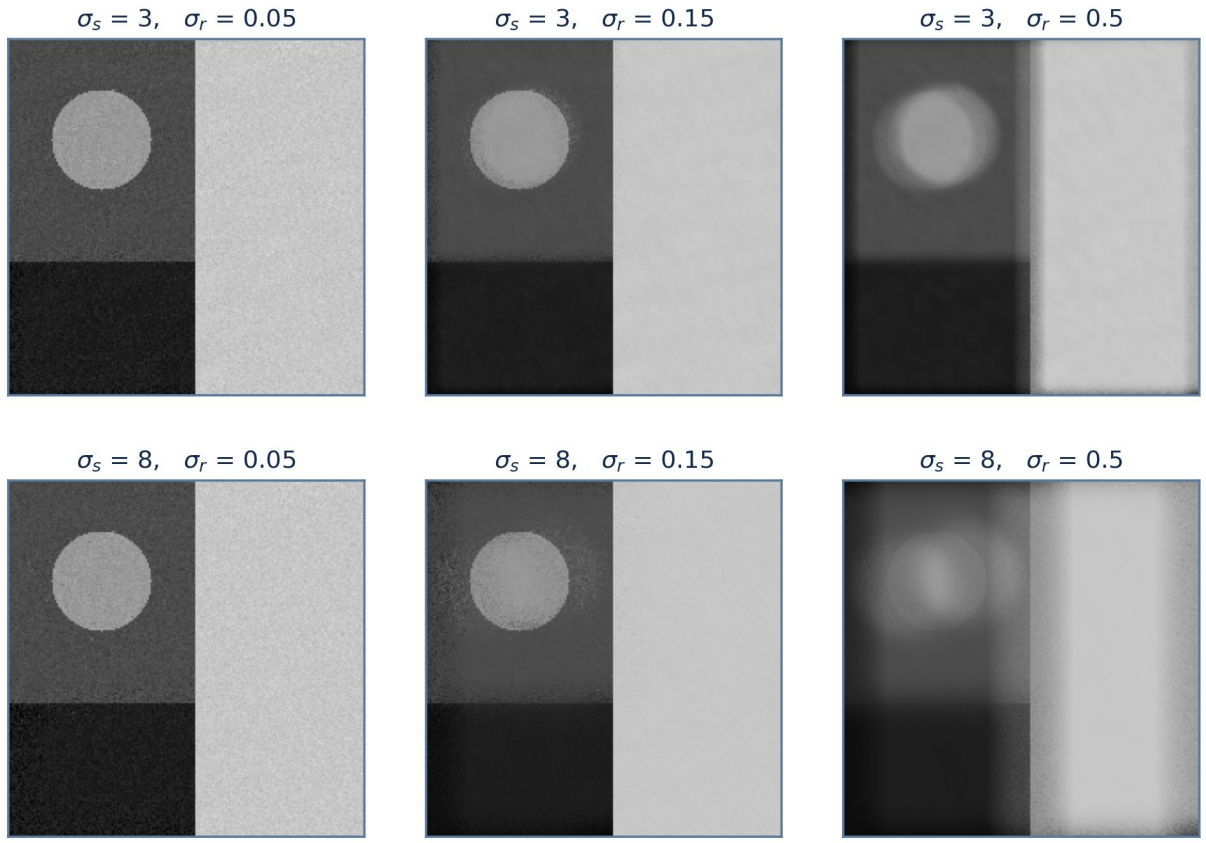
The reference pixel (red) sits just to the right of an edge. The space term alone happily reaches across it. The range term alone accepts every pixel of similar brightness. Multiplied together, only the near pixels on the same side of the edge survive.

The Window Reshapes Itself to the Image



The same construction in 2-D. Where a Gaussian would apply an identical set of weights everywhere, the bilateral weights are cut off at the boundary — so the filter simply never averages across it.

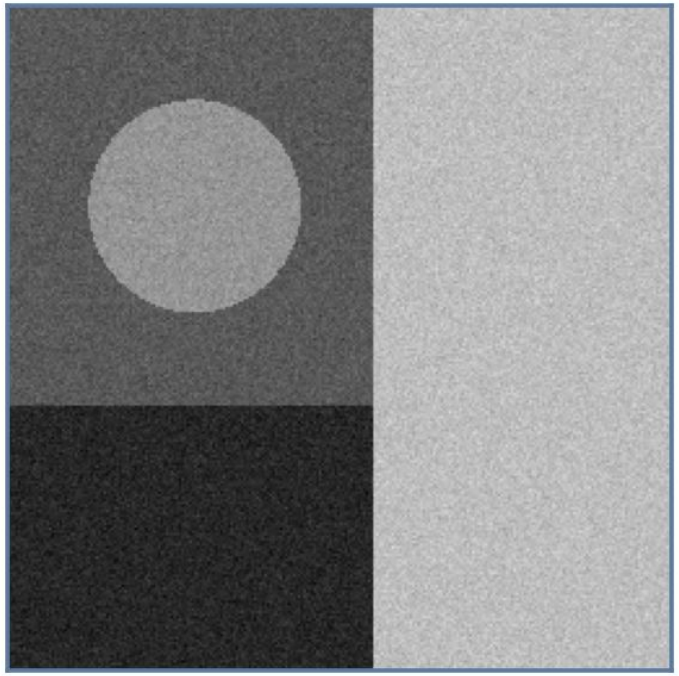
Choosing σ𝑠 and σ_r



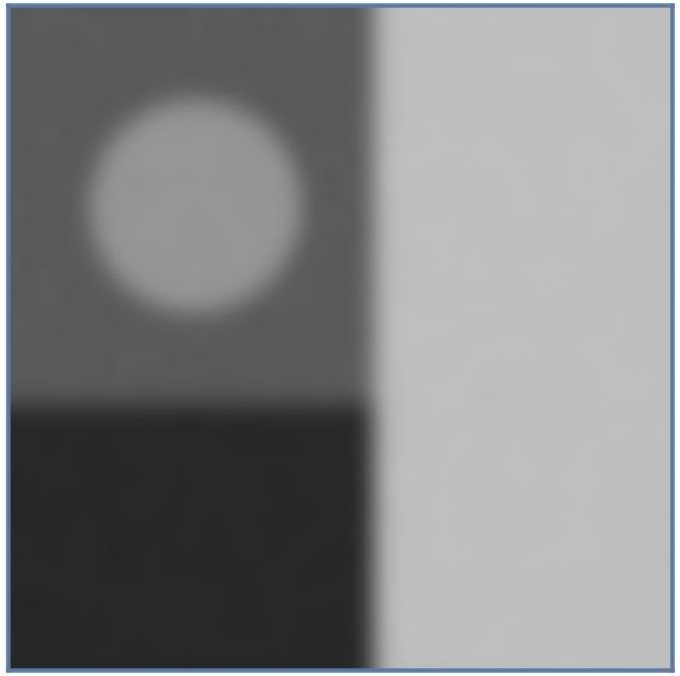
σ𝑠 sets how large a neighbourhood is considered; σ_r sets how different a pixel may be before it stops counting. As σ_r grows the range term stops discriminating and the result collapses back toward an ordinary Gaussian blur.

Bilateral Filtering in Action

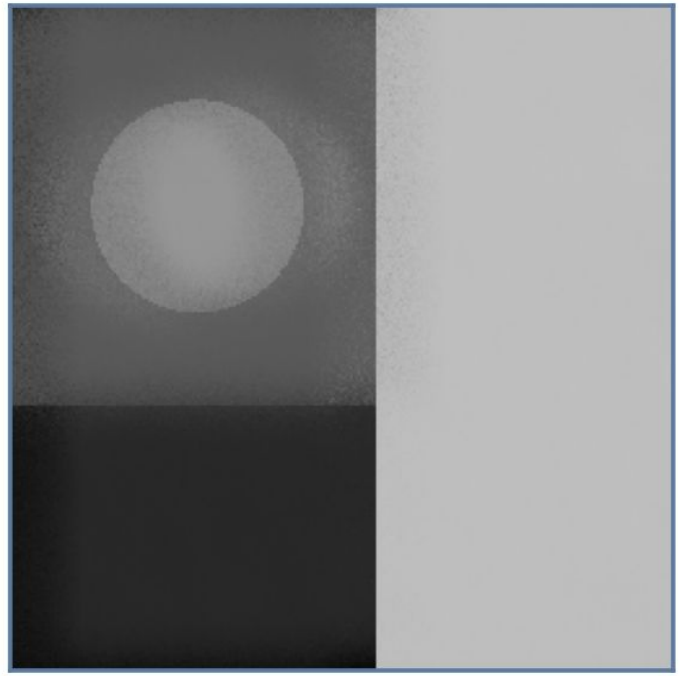
Original (noisy, sharp edges)



Gaussian filter
(smooths noise AND edges)

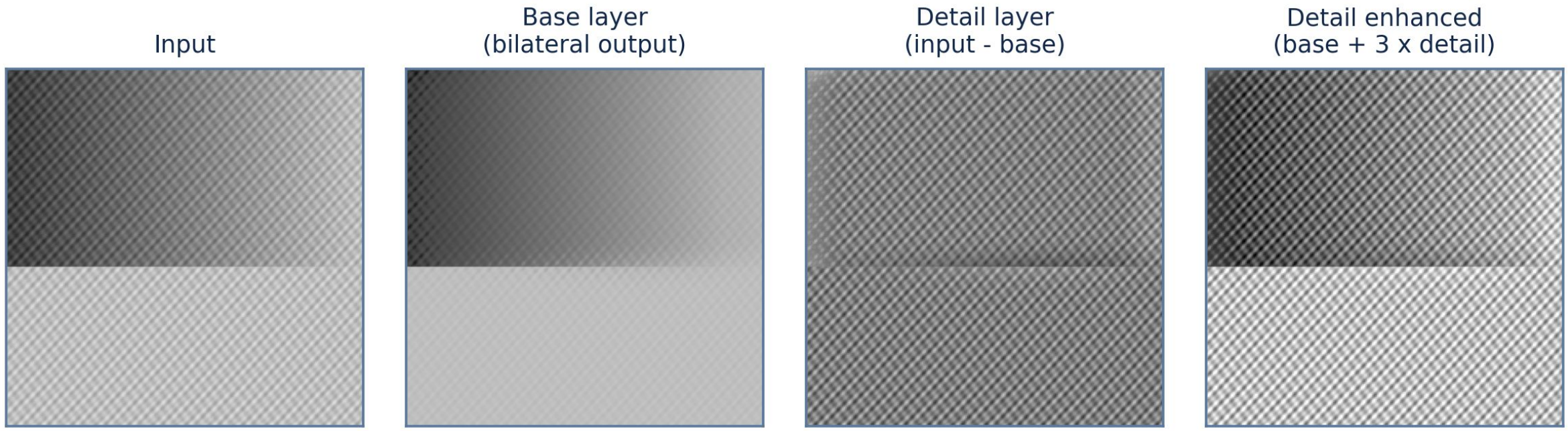


Bilateral filter
(smooths noise, keeps edges)



Both filters clean up the noisy flat regions. Only the bilateral filter keeps the boundaries between them sharp — because it refuses to average across a large intensity jump.

Splitting an Image into Base and Detail



Because the bilateral output keeps large-scale structure while dropping fine texture, subtracting it isolates the detail. Recombining the two with different gains enhances texture — the same decomposition underlies bilateral tone mapping for high-dynamic-range images.

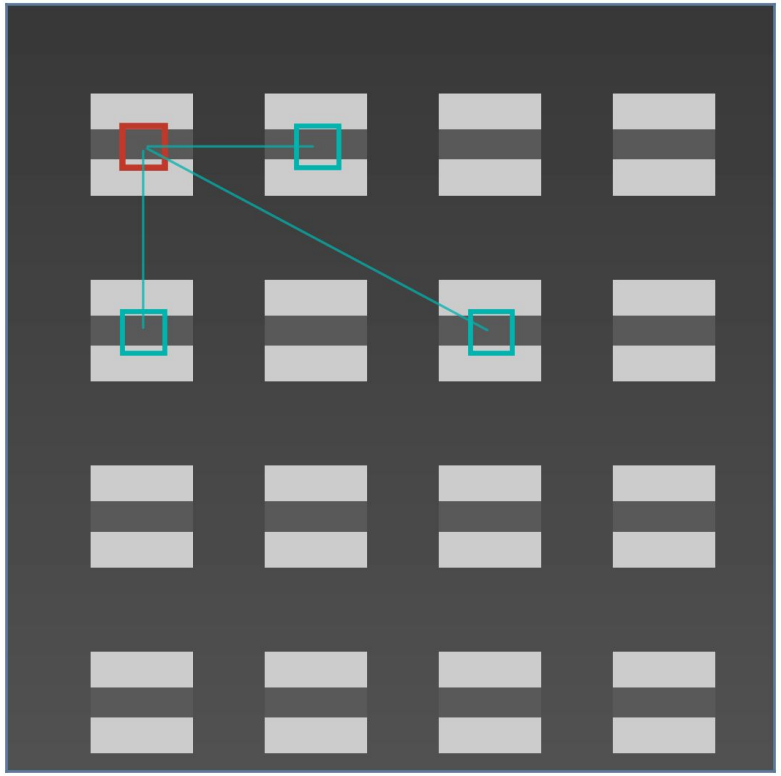
PART V

Non-Local Means

Judging neighbours by their surroundings, anywhere in the image

Natural Images Repeat Themselves

The same local pattern recurs
all over a natural image



Structure recurs

Brickwork, windows, fabric, foliage, even skin texture — the same small pattern shows up again and again in one image.

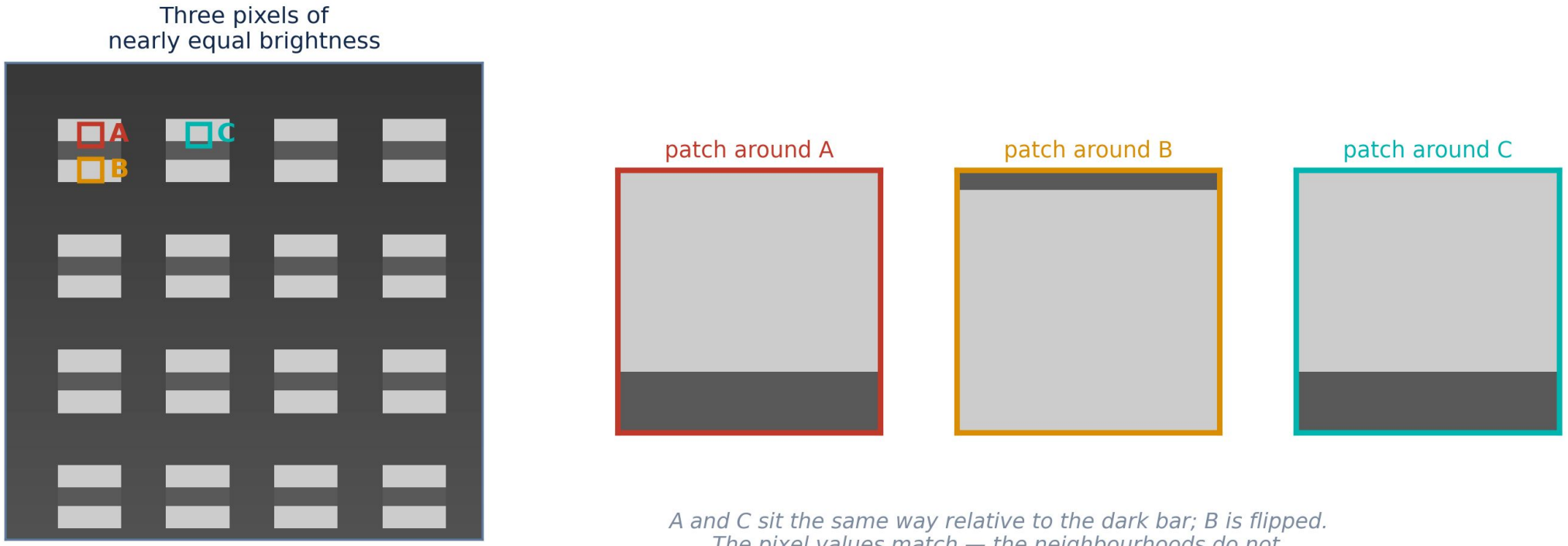
The repeats are far apart

The best match for a patch is often nowhere near it. Restricting ourselves to a small window throws that information away.

Noise does not repeat

The structure is shared across the copies; the noise is independent. Averaging matched patches cancels the noise and keeps the structure.

Compare Neighbourhoods, Not Pixels



A and C sit the same way relative to the dark bar; B is flipped.
The pixel values match — the neighbourhoods do not.

The bilateral range term compares two pixels by their own values alone — and under noise, that comparison is unreliable. Comparing the whole patch around each pixel is far more robust, and it distinguishes positions that a single value cannot.

DEFINITION

Non-Local Means

Same weighted average as before. The weight now compares the patch around p with the patch around q.

$$w(p,q) = \exp\left(- \| P(p) - P(q) \|^2 / h^2 \right)$$

P(p) = the k x k patch centred at p · h = how strict the match must be

Non-local

The search window is large — ideally the whole image — so distant repeats can contribute.

Patch-based

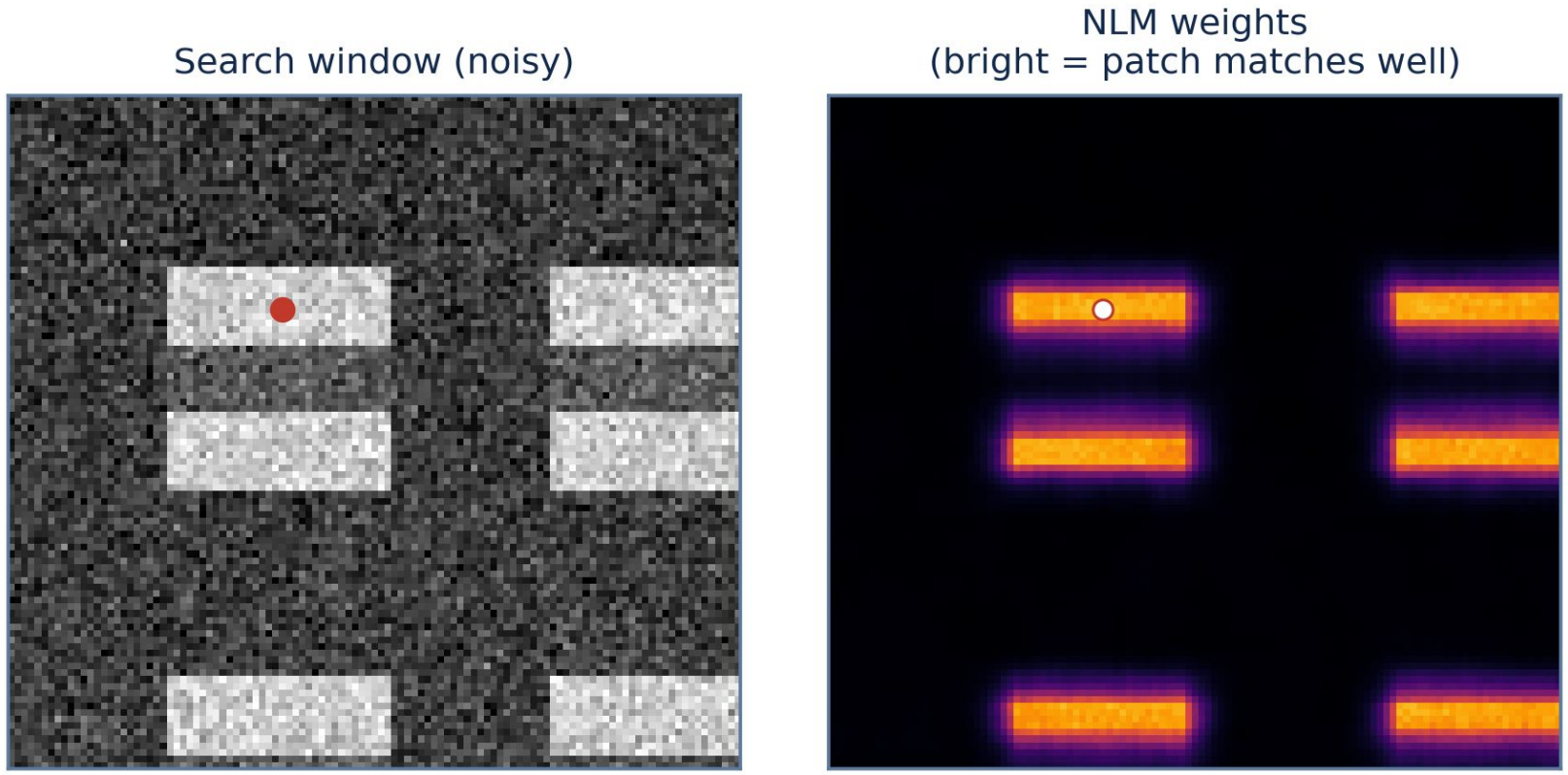
Similarity is measured over a whole neighbourhood, which averages out the noise in the comparison itself.

Expensive

Every candidate requires a full patch comparison. This is the price of the extra robustness.

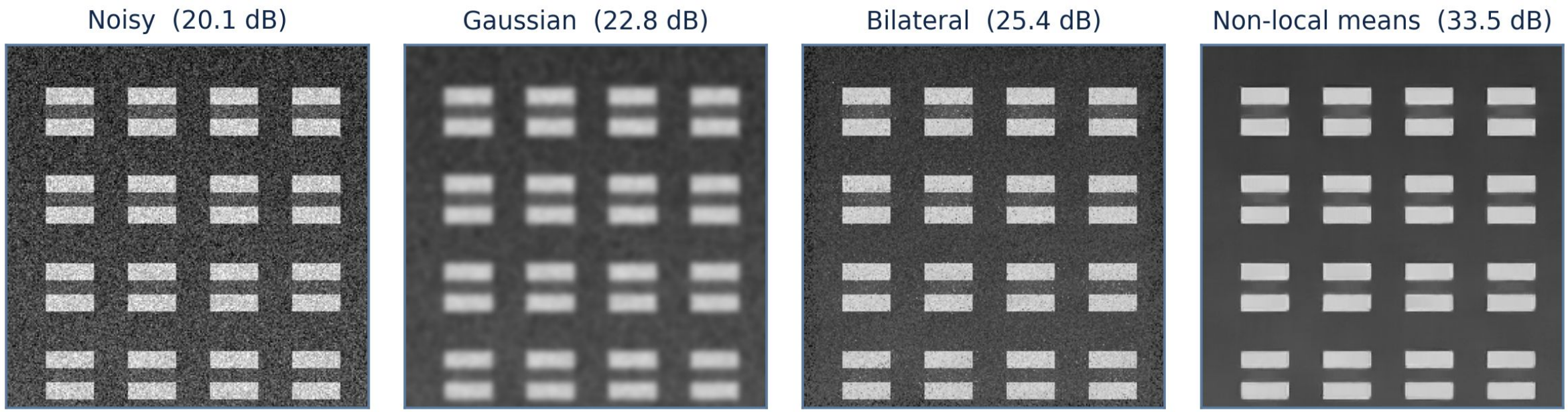
Buades, Coll & Morel, 2005

Where the Weight Actually Goes



Weights for one reference pixel across its search window. Every repeat of the same local pattern lights up — including copies far from the reference — while everything else contributes essentially nothing.

Gaussian vs. Bilateral vs. Non-Local Means



On an image with strong repeating structure, non-local means recovers detail the others cannot: the Gaussian blurs everything, the bilateral keeps edges but still only borrows from close neighbours, and NLM pools evidence from every repeat.

AN EXTENSION

Non-Local Patch Regression

NLM's weighted average is the patch P that minimizes $\Sigma\, w(p,q)\ \|P - P(q)\|^2$ — an ordinary least-squares fit in patch space.

$$\text{minimize}\ \ \Sigma\ w(p,q)\,\|P - P(q)\|^p \qquad (0 < p \le 2)$$

Why change the exponent?

Squaring punishes large residuals hardest, so a few badly-matched patches can drag the estimate away. A smaller exponent reduces their pull.

The outliers are structure

Near an edge, the mismatched patches come from the other side of it — they are structure, not noise. Down-weighting them is exactly what we want.

When it helps

At low noise, p near 2 is best. Past a certain noise level, smaller p wins — the weights become unreliable and robustness matters more.

Chaudhury & Singer, ICASSP 2013

Smoothing Filters at a Glance

Filter	Weights depend on	Keeps edges?	Cost	Typical use
Box (mean)	nothing — all equal	No	Very low	Fast, crude blur
Gaussian	distance from centre	No	Low	General-purpose smoothing
Binomial	distance (integer approx.)	No	Very low	Cheap, integer-only Gaussian
Bilateral	distance + intensity gap	Yes	Moderate	Denoising, detail and tone work
Non-local means	patch similarity, anywhere	Yes	High	Noisy images with repeated structure

Reading down the table, each row lets the weights depend on more of the image than the row above it.

NEXT CLASS

Sharpening: High-Pass Spatial Filtering

Derivatives, the Laplacian, and unsharp masking — keeping exactly what we threw away today.

Questions?